Laravel and PHP workshop

Index

Install XAMP (to create a local host server) and VS code.

Writing first PHP script:

Code:

```
<?php

echo'Hello, I am Nazmul';

?>
```

<?php ?> is the starting tag of php. Echo is the print of php. This can be run by using apaches admin dashboard by writing the path like localhost/projects/hello.php or use php server extenstion to run directly using a port.

Output:

Hello, I am Nazmul

Php variables:

Code:

```
<?php
$student = 'Nazmul';
$student2 = 'Aminul';
$question = ',What did you eat?';
echo 'Hello,'. $student;
echo '<br>';
echo 'How are you?';
echo '<br>';
echo $student.'Did you have dinner';
echo '<br>';
echo $student.$question;
echo '<br>';
echo 'Hello this is from another echo';
```

```
echo '<br>';
print 'Hello this is from print';
echo '<br>';
print "Hello $student2 this is from print";
$bool = true;
print "<br>$bool";
?>
```

$variable is the syntax. Variable data type is determined by value assigned like python. Dot (.) is used to join the strings or concatenation of two strings.
 is used to print in new line. Print can be also used like echo and values can be printed in double quotation only not single quotation. If bool is true then output is 1.

Output:

Hello,Nazmul

How are you?

NazmulDid you have dinner

Nazmul,What did you eat?

Hello this is from another echo

Hello this is from print

Hello Aminul this is from print

1

Php operators:

Code:

```
<?php
$x = 2;
$y = 3;
echo 'Value1 = ',$x;
echo '<br>Value2 = ',$y;
$result = $x + $y;
print "<br>$result";
$result = $x - $y;
print "<br>$result";
$result = $x * $y;
print "<br>$result";
$result = $x / $y;
print "<br>$result";
$result = $x % $y;
print "<br>$result";
$result = $x ** $y;
print "<br>$result";
?>
```

Operators mostly work like any other programming language.

Output:

Value1 = 2

Value2 = 3

5

-1

6

0.66666666666667

2

8

Operators:

Operators	Operation
+	Summation
-	Subtraction
*	Multiplication
/	Division
%	Modulus
**	x to the power y

==	Equal
===	Identical. Returns true if both are same data type
!=	Not equal
!==	Not identical. Returns true if both are different data type
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
?:	Ternary operator. $x condtion ? if true : else false

Php conditional statements:

Code:

```
<?php
$age = 50;
if ( $age<30) { // returns true if condition is correct.
   print 'You are under 30';
} else {
```

```
   print 'You are above 30';
}
if ($age > 10) $b = "Bigger than 10";
echo'<br>',$b;
$c = $age > 20 ? 'Your older than 20' : 'Your younger than 20';
echo '<br>',$c;
$a = 30;
if ($a <10){
   print "<br>It is smaller than 10";
}
elseif ($a < 20){
   print"<br>It is smaller than 20";
}
else {
   print"<br>It is bigger than 10 and 20";
}
$favcolor = "black";
switch ($favcolor) {
  case "red":
   echo "<br>Your favorite color is red!";
```

```
  break;
 case "blue":
  echo "<br>Your favorite color is blue!";
  break;
 case "green":
  echo "<br>Your favorite color is green!";
  break;
 default:
  echo "<br>Your favorite color is neither red, blue, nor green!";
}
?>
```

if else, elseif and switch works like any other programming language. There is short hand if and short hand if else like ternary operator ? : . If no case is matched in switch, then default works.

Output:

You are above 30

Bigger than 10

Your older than 20

It is bigger than 10 and 20

Your favorite color is neither red, blue, nor green!

Php loops:

Code:

```
<?php
// for loop
echo 'for loop:';
for ($i = 0; $i <10; $i++){
   print "<br>$i";
}
// while loop
echo '<br>while loop:';
$i = 10;
while ($i < 20){
   print"<br>$i";
   $i ++;
}
// do while loop
echo '<br>do while loop:';
$i = 20;
do {
   echo'<br>',$i;
   $i ++;
```

```
} while ($i <19);

// foreach loop

echo '<br>foreach loop:<br>';

$colors = array("red", "green", "blue", "yellow");

foreach ($colors as $x) {

  echo "$x <br>";

}

?>
```

for, while, do- while and foreach loop works like any other programming language. Break can be used to stop the loop and continue can be used to skip the iteration and move to the next one. In, do-while loop at least one iteration happens even if condition is false. In, foreach loop each element of an array is printed in each iteration.

Output:

for loop:

0

1

2

3

4

5

6

7

8

9

while loop:

10

11

12

13

14

15

16

17

18

19

do while loop:

20

foreach loop:

red

green

blue

yellow

Basic php in HTML:

Code:

```
<!DOCTYPE html>
<html>
<head>
<title> Page title</title>
</head>
<body>
<?php
$text = "Learning php";
print "<h>$text</h>";
?>
<h1> This is a heading tag.</h1>
<p>
    this is a paragraph of html.this is a paragraph of html.this is a paragraph of html.
    this is a paragraph of html.this is a paragraph of html.this is a paragraph of html.
    this is a paragraph of html.this is a paragraph of html.this is a paragraph of html.
</p>
```

```
</body>

</html>
```

<!--comment --> is used to comment in HTML. <!DOCTYPE html> this is the start tag of HTML. <html>

HTML start tag, </html> HTML end tag. <head> marks head, </head> ends head. <title> and </title> gives page name shown in tab bar. <body> start of the body of web page, </body> end of the body of webpage. <?php ?> can be called and used in the body. <h1> to create a heading title in the body, </h1> to end the heading. is used to start the paragraph and, to end the paragraph.

Output:

Learning php

This is a heading tag.

this is a paragraph of html.this is a paragraph of html.this is a paragraph of html. this is a paragraph of html.this is a paragraph of html.this is a paragraph of html. this is a paragraph of html.this is a paragraph of html.this is a paragraph of html.

Php functions:

Code:

```
<?php

// functions in php
```

```
function familyName($fname,int $year) {
  echo "$fname Refsnes. Born in $year <br>";
}
familyName("John", 1975); // first parameter is John and second parameter is 1975
familyName("Max", 1978);
// default arguement
function set_height($height = 50){
   echo "The height is : $height <br>";
}
set_height(350);
set_height();
// return type function
function sum($x, $y) {
  $z = $x + $y;
  return $z;
}
echo "5 + 10 = " . sum(5, 10) . "<br>";
echo "7 + 13 = " . sum(7, 13) . "<br>";
// function accepting unknown number of arguements
function sumMyNumbers(...$x) {
```

```
  $n = 0;
  $len = count($x);
  for($i = 0; $i < $len; $i++) {
    $n += $x[$i];
  }
  return $n;
}
$a = sumMyNumbers(5, 2, 6, 2, 7, 7);
echo $a,'<br>';
// two arguements with unknown number
function myFamily($lastname, ...$firstname) {
    $txt = "";
  $len = count($firstname);
  for($i = 0; $i < $len; $i++) {
    $txt = $txt."Hi, $firstname[$i] $lastname.<br>";
}
    return $txt;
}
$b = myFamily("Doe", "Jane", "John", "Joey");
echo $b;
?>
```

function keyword is used to define a function in php which is different from other programming languages. It can take multiple arguments or parameters and parameter data type can be pre-defined. Arguments can be set with a default value like other programming languages. It can also return values. (...) operator is used before the last argument to define that function can take unknown number of arguments.

Output:

John Refsnes. Born in 1975

Max Refsnes. Born in 1978

The height is : 350

The height is : 50

5 + 10 = 15

7 + 13 = 20

29

Hi, Jane Doe.

Hi, John Doe.

Hi, Joey Doe.

Php array:

Code:

<?php

```
// Creating array
$cars0 = array('volvo', 'toyota', 'bmw');
// another way to create array using []
$cars = ['volvo', 'toyota', 'mitsubishi'];
echo $cars[1],'<br>';
// indexed arrays through a loop
foreach ($cars as $x) {
  echo "$x <br>";
}
// associative arrrays
$cars = ["brand"=>"Ford", "model"=>"Mustang", "year"=>1964];
echo $cars["model"];
$cars["year"] = 2024;
echo '<br>';
var_dump($cars);
echo '<br>';
$car = ["brand"=>"Ford", "model"=>"Mustang", "year"=>1964];
foreach ($car as $x => $y) {
  echo "$x: $y <br>";
```

```
}
// multiple line declaration of array
$myCar = [
  "brand" => "Ford",
  "model" => "Mustang",
  "year" => 1964
];
echo $myCar["brand"];
// adding multiple items in array
$fruits = ["Apple", "Banana", "Cherry"];
array_push($fruits, "Orange", "Kiwi", "Lemon");
echo '<br>';
var_dump($fruits);
$cars += ["Speed" => 160];
echo '<br>';
var_dump($cars);
// deleting items from array
$car = ["volvo", "lamborghini", "bmw", "toyota", "mitsubishi","ferrari"];
array_splice($car, 1, 2);
echo '<br>';
```

```
var_dump($car);
unset($car[1]);
echo '<br>';
var_dump($car);
// remove items from an associative array
unset($cars["Speed"]);
echo '<br>';
var_dump($cars);
$car = ["volvo", "lamborghini", "bmw", "toyota", "mitsubishi","ferrari"];
array_pop($car);
echo '<br>';
var_dump($car);
array_shift($car);
echo '<br>';
var_dump($car);
?>
```

array can be declared by array() or using square brackets []. This works like list in python. Associative arrays are like dictionary in python and it can be multi-lined, using trailing commas. By using foreach associated_array as $x => $y we can access the associates its values. Using array_push() we

can add multiple array items after the last index. Var_dump() is used to print our array. We can also use += operator to add new elements in associated array, this only works in associated array. By using array_splice(, starting index, number of items to be deleted) we can delete one or multiple elements at specific index. And unset() method deletes the item but does not rearranges the array so the index remains empty and it is also used to delete items in associated array. Using array_pop() deletes the last element of the array and array_shift() deletes the first element of the array.

Output:

toyota

volvo

toyota

mitsubishi

Mustang

array(3) { ["brand"]=> string(4) "Ford" ["model"]=> string(7) "Mustang" ["year"]=> int(2024) }

brand: Ford

model: Mustang

year: 1964

Ford

array(6) { [0]=> string(5) "Apple" [1]=> string(6) "Banana" [2]=> string(6) "Cherry" [3]=> string(6) "Orange" [4]=> string(4) "Kiwi" [5]=> string(5) "Lemon" }

array(4) { ["brand"]=> string(4) "Ford" ["model"]=> string(7) "Mustang" ["year"]=> int(2024) ["Speed"]=> int(160) }

array(4) { [0]=> string(5) "volvo" [1]=> string(6) "toyota" [2]=> string(10) "mitsubishi" [3]=> string(7) "ferrari" }

array(3) { [0]=> string(5) "volvo" [2]=> string(10) "mitsubishi" [3]=> string(7) "ferrari" }

array(3) { ["brand"]=> string(4) "Ford" ["model"]=> string(7) "Mustang" ["year"]=> int(2024) }

array(5) { [0]=> string(5) "volvo" [1]=> string(11) "lamborghini" [2]=> string(3) "bmw" [3]=> string(6) "toyota" [4]=> string(10) "mitsubishi" }

array(4) { [0]=> string(11) "lamborghini" [1]=> string(3) "bmw" [2]=> string(6) "toyota" [3]=> string(10) "mitsubishi" }

Laravel installation:

Put php in environment variables path while xamp running. Download and install composer. Run commans to install Laravel. Edit php file in xamp folder to enable xamp extention-zip. Click on desired folder and open in terminal to run:

laravel new my-app

This creates a new project in that field. Choose livewire to use blade files. Then choose volt and pest.

File Structure:

- Http contains the user controller file and model file. Resource contains the view that contains the blade file. Blade file is the view or html file for the website.
- Routes contains the web.php for routing.
- Storage and public can contain user data like image.
- Env file is used for configuration like database connection through my sql, apps url, mail access port key etc.
- Database file is used for containing the database tables for migration.

Basic Artisan Commands:

Open project file location then click and open in terminal.

To run the project on a port:

php artisan serve

To make a controller:

php artisan make:controller controller name

To make a model:

php artisan make:model model name

To see php artisan commands:

php artisan list

Blade file:

Creating a blade file syntax filen-ame.blade.php . In a blade file, <?php ?> tag is not needed. We can directly write if else just by adding @ before the if else like @if, @else, @for, @foreach, @include, @extendsn, @section, for displaying variables {{ $name}} etc. This is called short cut php.

Routes:

It is used to traverse path. Basically, moves one page view to another through routing.

Code:

```
<?php
use Illuminate\Support\Facades\Route;
use Livewire\Volt\Volt;
Route::get('/', function () {
   return view('index');
})->name('home');
Route::get('/portfolio', function () {
   return view('portfolio_view');
});
```

Here we can route to home page to portfolio page by traversing means typing ‘/portfolio’ in the url. This takes to the view of portfolio_view file.

Workshop:

Create a new project on clicking the desired folder and open in terminal to run this command:

laravel new my-app

Connect Databse:

Open xamp turn on apache and mysql. Then go to configuration file .env and find the DB connection, remove the comments:

DB_CONNECTION=mysql

DB_HOST=127.0.0.1

DB_PORT=3307

DB_DATABASE=taskmanager

DB_USERNAME=root

DB_PASSWORD=

Rename it to mysql. Set the port by default its 3306, but I have mine on 3307. Then create a database named taskmanager in the chrome browser admin panel. Set the name of the database in configuration file.

Checking Database Connection:

Run this command:

php artisan migrate

There are default tables in database file. By running that command those tables will be added to database. If it shows cache, jobs and user the database is successfully connected.

Home Page:

Create home.blade.php in view file and write the code of a basic bootstrap template. Then, go to route file and open web.php and set home to route it to home page.

Make Model and Migrate:

Run this command:

php artisan make:model Task -m

To create a model named Task.php in apps file, model file. It maintains our table connection with database. After creating the model -m migrates it or uploads it to database. The migration file named create_tasks_table.php is created in the database file.

Create Columns in Table:

Open the create_tasks_table.php in database file. Then edit the schema create( ). To add these:

```
$table->string('title');
$table->text('description')->nullable();
$table->boolean('is complete')->defautl(false);
```

Then run this command to migrate this table to database:

```
php artisan migrate
```

Finally check the admin panel of mysql to confirm if its added or not.

Write Code in Model:

Open Task.php model file and in class Task add code for fillables like title, description and is_complete. Since we created table using command so by default table is connected to the model we don't have to call it.

Make Controller:

Run this command:

```
php artisan make:controller TaskController
```

Creates a controller named TaskController.php in app files, http file.

Create Views:

Create a folder named task in view file. Inside task folder create index.php, edit.php, create.php, view.php files.

Create a Name Route:

Go to routes and use a get, a class and a function or method to create name route. This name route can be used in views to redirect to that path in route. Export controller in the route.

Create Function in Controller:

Create the index function which can be used in route. Must import the Task model in controller.

Insert Dummies in DB:

Go to mysql admin panel and from insert add dummy tilte in tile and dummy des 1 in description. Insert dummy tile 2 and dummy des 2.

Code for Index:

Write code for index view use @foreach and table tag. End with @endforeach.

Code for Route:

Write all the route get, post (is used to store data) and delete code for routing. {{id}} parameter is used to find which one to show, edit or delete.

Create Functions in Controller:

Create necessary functions that was written in route like create, edit, show and delete. Also import StoreTaskRequest.php from requests file for validation.

Create Request for validation:

Run this command:

php artisan make:request StoreTaskRequest

To create a StoreTaskRequest.php in requests file for validation when adding a new task or editing it. In public

function authorize ( ) edit return and change it to true. In public function rules ( ) edit return and add these codes:

title' => 'required|string|max:255',

'description' => 'nullable|string',

Write Code for Views:

Type the codes for index, home, create, edit, view, about blade file. Use:

href="{{ route(' named_route') }}"

to traverse or go to another page, pair it up with a button tag. Use table tags with @foreach, @endforeach, form tag, input tag, @error and @enderror in create blade file. Use @if , @endif in index blade file.

Issues:

In mysql is_complete may not set up with a default value. In mysql admin, just go to table then structure and select it, and change the default value to null.